

Table S1. Training settings of the two models

Graph regressor and perceptron, 60 runs each at $g = 5$ km; the runs at $g = 10$ km (Table 7 of the article) use the same settings, with only the block side changed. “same” marks a value identical in the two models. Architecture and loss (first half; the optimisation, selection, baselines and cost follow on the next page).

Setting	Graph regressor	Perceptron
<i>Architecture</i>		
Layers	4 (message passing)	5 (encoder)
Neighbourhood	one hop per relation, full neighbourhood (no sampling)	node batches, no graph
Operators	transmitter→terrain and terrain→terrain (Moore neighbourhood with self-loop): GATv2, 4 heads concatenated, edge dimension 2; terrain→transmitter: SAGE with mean aggregation, no gradient	none (no edges)
Relation aggregation	sum	–
Layer	LeakyReLU (slope 0.2), dropout, residual connection, LayerNorm	linear, BatchNorm, LeakyReLU (slope 0.2), dropout (not after the last block)
Attention heads	4	–
Width	256 (hidden)	654, 654, 600, 636, 256 (encoder)
Input features per terrain node	18	same
Input features per transmitter	6	not used
Output channels	5	same
Dropout	0.1	same
Parameters, nominal	1 812 515	1 484 067
Parameters receiving gradient	1 484 037	same
Width-sensitivity arm (10 runs)	–	widths 712, 712, 720, 688, 256; 1 812 515 parameters
<i>Loss</i>		
Prediction term	Huber loss with threshold 5 dB on each of the 5 channels, averaged over the seed nodes (sentinel nodes included)	same
Channel weights	uniform, 1/5 each (fixed)	same
Distance-gradient penalty	weight 0.05; $\text{mean}_{(i,j): \text{dist}_i < \text{dist}_j} \max\{0, \hat{P}_j - \hat{P}_i + 1 \text{ dB}\}$ over K node pairs drawn in each batch, K being the smaller of 512 and half the batch size rounded down, dist being the distance to the nearest transmitter	same
Variance penalty	weight 0.02; $\max\{0, \text{sd}(P) - \text{sd}(\hat{P})\}^2$ over the batch, target without gradient	same
Shadowing×NDVI penalty	weight 0.03; $\max\{0, 0.15 - \text{corr}(\hat{L}_{\text{veg}}, \text{NDVI})\}$ over the batch, NDVI without gradient, $\hat{L}_{\text{veg}}$ being the vegetation component of the decoded path loss; identically zero in these runs	same
Free-space loss penalty	weight 0.05; $\text{mean} \max\{0, \text{FSPL}_{900}(\text{dist}) - \hat{L}_{\text{tot}}\}$ over the batch, $\hat{L}_{\text{tot}}$ being the decoded total path loss	same

Table S1 (continued)

Setting	Graph regressor	Perceptron
<i>Optimisation</i>		
Optimiser	AdamW, weight decay 10^{-5}	same
Initial learning rate	10^{-3}	same
Learning-rate schedule	cosine annealing with warm restarts, $T_0 = 10$ epochs, $T_{\text{mult}} = 2$, minimum rate 10^{-6} , stepped per epoch; no restart falls within the 8 epochs	same
Gradient clipping, norm threshold	0.5	same
Mixed precision	automatic mixed precision (CUDA)	same
Batch size (nodes)	12288	same
Epochs	8	same
Early stopping	none	same
Gradient steps per run, median (range)	5912 (5880–5920)	same
<i>Selection</i>		
Epoch retained	lowest validation MAE of received power	same
Test set used in selection	no	same
Best epoch, median (range)	6 (2–8)	6 (1–8)
<i>Baselines</i>		
Frequency (MHz)	900	same
Heights (m)	Hata rural and COST-231: $h_{\text{tx}} = 30$, $h_{\text{rx}} = 1.5$	same
COST-231 variant	suburban ($C_m = 0$)	same
Distance	to the nearest transmitter, floored at 1 m	same
Calibration	offset equal to the training median of received power plus modelled path loss, over (a) all training nodes, (b) valid training nodes only	same
<i>Hardware and cost</i>		
Hardware and software	NVIDIA GeForce RTX 5070 Ti; PyTorch 2.10.0+cu128; CUDA 12.8	same
Time per run (s), median (range)	783.05 (775.9–1141.3)	84.55 (82.5–192.2)
Peak GPU memory (MB), median (range)	4159.9 (4140.7–4187.6)	316.3